

19 Dynamische Datenstrukturen



Um Daten geordnet im Speicher zu halten, werden in der Programmiersprache C die Strukturen (Kapitel [→ 13](#)) verwendet. Kombiniert man diese mit der dynamischen Speicherzuweisung (Kapitel [→ 18](#)), so ist es damit möglich, beliebig viel Speicher für beliebige Datenblöcke zu reservieren.

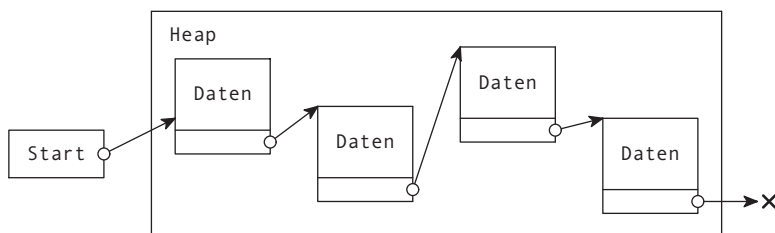
Das Ansprechen solcher Speicherblöcke erfolgt über eine Adresse. Um somit Strukturen mittels dynamischer Speicherzuweisung zu verwalten, müssen insbesondere die Pointer zu diesen Blöcken gespeichert und verwaltet werden.

Bei sehr vielen Datenblöcken kann es dabei schwierig werden, den Überblick über alle gespeicherten Datenblöcke zu bewahren. Aus diesem Grund gibt es die sogenannten „**dynamischen Datenstrukturen**“, welche die Aufgabe haben, die dynamischen Speicherblöcke in Relation zueinander geordnet abzuliegen.

In diesem Kapitel werden insbesondere zwei dieser dynamischen Datenstrukturen vorgestellt: die verkettete Liste und die Baumstruktur. Beide Datenstrukturen erlauben es, je nach Bedarf Datenblöcke geordnet hinzuzufügen, sie zu durchsuchen, oder wieder wegzunehmen.

19.1 Verkettete Listen

Bei einer verketteten **Liste** stehen die **Elemente** nicht wie bei einem Array sequenziell hintereinander, sondern können beliebig im Heap verstreut sein. Da somit nicht sichergestellt ist, dass die Daten, welche auf dem Heap abgelegt werden, immer hintereinander stehen, muss man zum Referenzieren der Elemente entweder einen Pointer auf jedes Element anlegen (beispielsweise mittels eines Pointer-Arrays) oder aber die einzelnen Elemente so miteinander über Pointer verknüpfen, dass jeweils ein Element auf das nächste zeigt und es möglich ist, über einen Pointer auf ein Ausgangselement und über die Verknüpfungen zwischen den Elementen jedes einzelne Element wiederzufinden. Bei dieser zweiten Art der Ablage spricht man von einer verketteten Liste.



Ergänzende Information Die elektronische Version dieses Kapitels enthält Zusatzmaterial, auf das über folgenden Link zugegriffen werden kann https://doi.org/10.1007/978-3-658-45209-4_19.

Eine verkettete Liste besteht aus gleichartigen Datenelementen. Um von einem Listenelement zum nächsten zu finden, muss ein Listenelement einen Pointer auf das nächste Listenelement besitzen. Ein Listenelement besteht aus einem Datenteil mit Nutzdaten und einem Pointer, der zur Verkettung der Listenelemente dient.



Die Listenelemente werden im Heap angelegt. Sie können an beliebiger Stelle im Heap liegen. Wird ein Element in der Liste gesucht, so muss von einem Start-Pointer, der auf das erste Element der Liste zeigt, solange von einem Element zum nächsten gegangen werden, bis entweder das letzte Element der Liste erreicht oder das gesuchte Element gefunden ist.

19.1.1 Datentyp eines Listenelements

Ein Listenelement wird als eine Struktur definiert, wie beispielsweise

```
struct Element {  
    float f;  
    struct Element* next;  
};
```

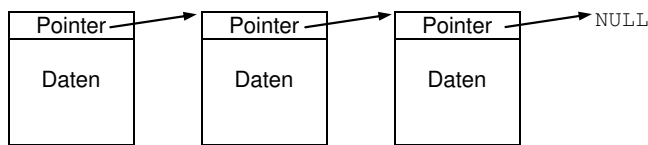
Man beachte, dass, obwohl der Typ `struct Element` noch nicht definiert ist, er innerhalb der Struktur bereits benutzt werden kann (`struct Element* next`). Dies ist möglich, da – ähnlich zur Vorwärtsdeklaration von Funktionen – der Compiler den Namen der Struktur bereits mit der Deklaration `struct Element` registriert. Siehe dazu Kapitel [→ 13.1.4](#)

19.1.2 Einfach verkettete Liste

Die einfachste Form einer verketteten Liste ist die Einfachverkettung. Eine einfache Verkettung besteht darin, dass jedes Element der Liste immer einen Nachfolger hat und auf diesen mit einem Pointer verweist.



Im Folgenden werden Elemente einer verketteten Liste symbolisiert:

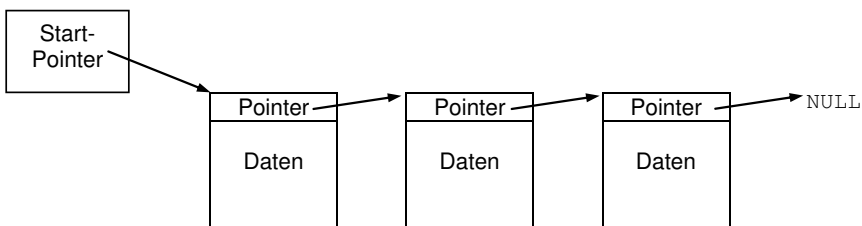


Jedes dieser Kästchen bezeichnet einen einzelnen Datenblock, welcher beispielsweise mittels der Funktion `malloc()` (siehe Kapitel [→ 18.1.1](#)) irgendwo im Heap reserviert wurde. Wo genau die einzelnen Datenblöcke stehen, ist irrelevant, da jedes Kästchen über seinen Pointer-Eintrag festhält, wo sich der nächste Datenblock befindet.

Das letzte Element hat keinen Nachfolger und bezeichnet somit das Ende der Liste. Um das Ende der Liste zu definieren, wird häufig der Pointer des letzten Elements auf NULL gesetzt.



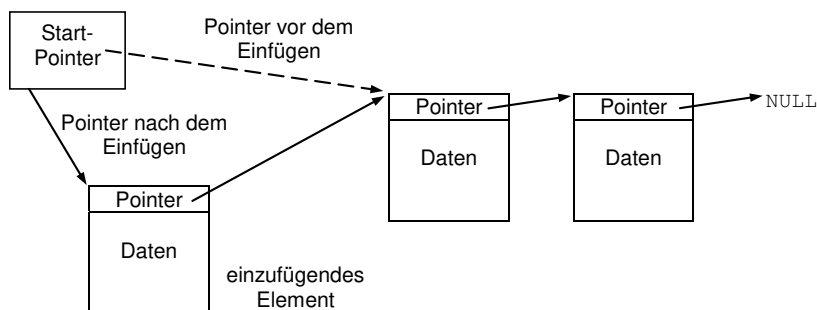
Um die gesamte Liste im Heap zu finden, muss jetzt lediglich ein Pointer auf das erste Element der Liste angelegt werden.



Um mit der Liste arbeiten zu können, werden insbesondere zwei Aktionen benötigt: Das Einfügen eines neuen Elements und das Entfernen eines Elements.

19.1.3 Neues Listenelement einfügen

Das folgende Bild zeigt das Einfügen eines Elements in eine einfach verkettete Liste:



Hier wird das neue Element am Anfang der Liste eingefügt: Dem Pointer des neu einzutragenden Listenelements wird die Adresse des bisher ersten Listenelements zugewiesen und dem Startpointer wird danach die Adresse des neuen Listenelements zugewiesen.

Ein neues Listenelement kann grundsätzlich an einer beliebigen Stelle in die bestehende Liste eingefügt werden. Ist jedoch die Adresse des Listenelements, an welchem das Einfügen passieren soll, nicht bekannt, so muss es zuerst innerhalb der Liste gefunden werden.

Das Suchen nach einem bestimmten Element innerhalb der Liste funktioniert so, dass man mit dem ersten Element der Liste beginnt und daraufhin jedes Element einzeln abarbeitet (möglicherweise bis zum letzten Element), bis das gewünschte Element gefunden wurde. Eine Suche auf einer einfach verketteten Liste hat somit kein gutes Laufzeitverhalten.

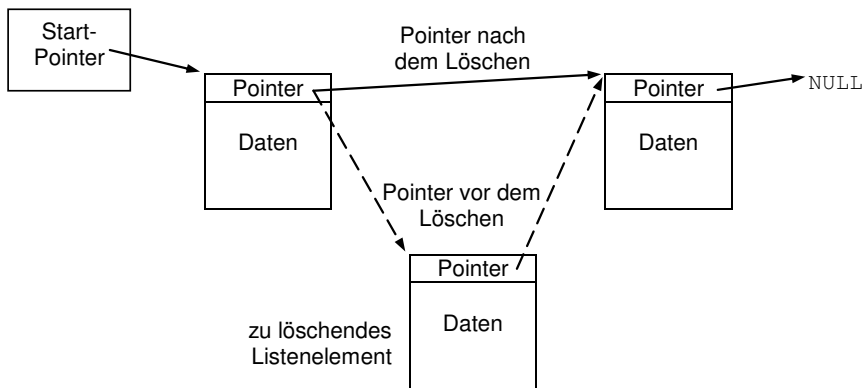
Operationen auf Elementen von einfach verketteten Listen, zu welchen die Adresse nicht bereits bekannt ist, sind sehr zeitaufwendig.



Siehe dazu auch Kapitel [→ 20](#) über Suchen und Sortieren.

19.1.4 Listenelement entfernen

Um ein Element aus einer einfach verketteten Liste zu entfernen, muss im ersten Schritt das entsprechende Element gesucht werden. Dann wird der Pointer des Vorgängers des zu entfernenden Elementes auf den Nachfolger des zu entfernenden Elementes gesetzt. Falls zudem das erste Element der Liste entfernt wird, muss der Start-Pointer berichtigt werden.



Es ist zu beachten, dass nach diesem Vorgang das Element zwar aus der Liste entfernt wurde, es jedoch dennoch im Heap Speicher belegt. Soll das Element nicht nur entfernt, sondern auch gelöscht werden, so muss das Element nach diesem Vorgang somit manuell gelöscht werden, beispielsweise mittels `free()`.

19.1.5 Programmbeispiel für eine einfach verkettete Liste

Hier ein Programmbeispiel für eine einfach verkettete Liste. Jedes Listenelement speichert einen Messwert und den Namen des entsprechenden Sensors.

liste.c

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

struct Messdaten {
    struct Messdaten* next;
    char    sensorname[30];
    double messwert;
};

struct Messdaten* startPoint = NULL;

struct Messdaten* allocMessdaten(const char* name, double messwert) {
    struct Messdaten* ptr = malloc(sizeof(struct Messdaten));
    strcpy(ptr->sensorname, name);

    ptr->messwert = messwert;
    ptr->next = NULL;

    return ptr;
}

void freeMessdaten(struct Messdaten* messdaten) {
    free(messdaten);
}

void elementHinzufuegen(struct Messdaten* messdaten) {
    messdaten->next = startPoint;
    startPoint = messdaten;
}

struct Messdaten* elementSuchen(const char* name) {
    struct Messdaten* ptr = startPoint;

    while (ptr != NULL && strcmp(ptr->sensorname, name)) {
        ptr = ptr->next;
    }

    return ptr;
}
```

```
struct Messdaten* elementEntfernen(struct Messdaten* messdaten) {
    struct Messdaten* ptr = startPointer;
    struct Messdaten* vorgaenger = NULL;

    while (ptr != messdaten) {
        vorgaenger = ptr;
        ptr = ptr->next;
    }
    if (ptr == NULL)
        return NULL;
    if (ptr == startPointer)
        startPointer = ptr->next;
    else
        vorgaenger->next = ptr->next;

    return ptr;
}

void elementeAusgeben(void) {
    struct Messdaten* ptr = startPointer;
    while (ptr != NULL) {
        printf("%s: %f\n", ptr->sensorname, ptr->messwert);
        ptr = ptr->next;
    }
    printf("\n");
}

int main(void) {
    elementHinzufuegen(allocMessdaten("Temperatur", 23.));
    elementHinzufuegen(allocMessdaten("Luftdruck", 1023.));
    elementHinzufuegen(allocMessdaten("Wasserpegel", 37.));
    elementeAusgeben();

    struct Messdaten* luftdruck = elementSuchen("Luftdruck");
    if (luftdruck)
        printf("Aktueller Luftdruck: %f\n\n", luftdruck->messwert);

    freeMessdaten(elementEntfernen(luftdruck));
    elementeAusgeben();

    while (startPointer) {
        freeMessdaten(elementEntfernen(startPointer));
    }

    return 0;
}
```

Die Funktion `main()` gibt als erstes die gesamte Liste aus, sucht danach nach dem Eintrag des Luftdrucks, gibt ihn als einzelnen Wert auf dem Bildschirm aus, löscht ihn aus der Liste und gibt die übrigbleibende Liste wieder aus.

Hier die Ausgabe des Programms:

```
Wasserpegel: 37.000000
Luftdruck: 1023.000000
Temperatur: 23.000000

Aktueller Luftdruck: 1023.000000

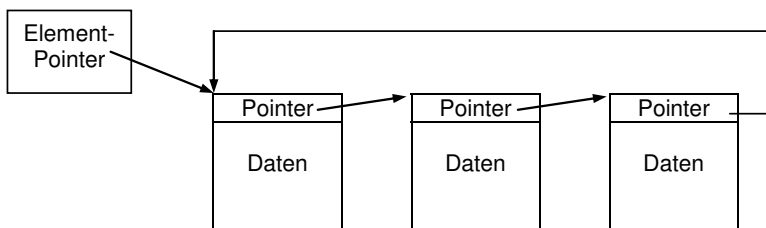
Wasserpegel: 37.000000
Temperatur: 23.000000
```

Beachten Sie, dass die eingegebenen Daten in umgekehrter Reihe ausgegeben werden, da die Elemente jeweils zu Beginn der Liste eingefügt werden.

19.1.6 Andere Listenarten

Neben einer einfach verketteten Liste gibt es noch verschiedene andere Listenarten wie beispielsweise Ringstrukturen und doppelt verkettete Listen.

Belegt man das letzte Element einer einfach verketteten Liste nicht mit dem Wert NULL, sondern verbindet dieses Element wieder mit dem Anfang der Liste, so ergibt sich eine Ringstruktur.



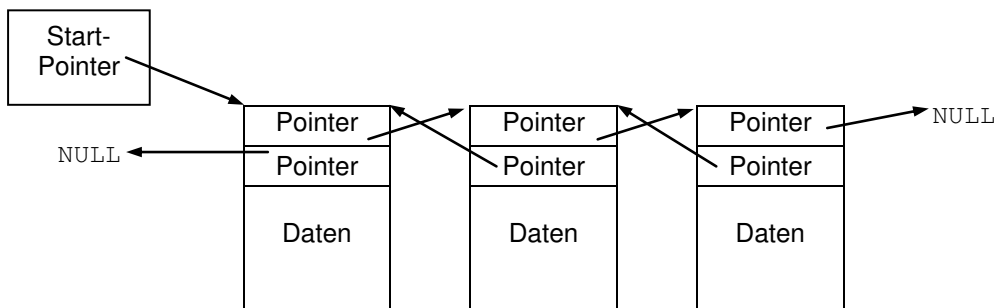
Um auf die Elemente zuzugreifen, muss ein Pointer auf ein beliebiges Element der Liste vorhanden sein. Egal welches Element verwendet wird, von ihm aus werden alle anderen Elemente des Rings angesprochen.

Da kein „einfaches“ Abbruchkriterium wie der NULL-Pointer bei einer linearen verketteten Liste vorhanden ist, muss beim Durchsuchen einer Ringstruktur insbesondere darauf geachtet werden, dass keine Endlosschleife programmiert wird.



Eine weitere Art von Listen sind solche, durch welche sowohl vorwärts, als auch rückwärts gegangen werden kann.

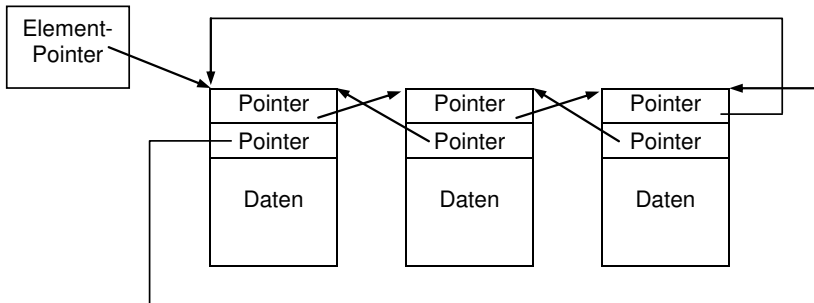
Speichert man in einem Listenelement außer der Adresse des Nachfolgers auch noch die Adresse des Vorgängers, so spricht man von einer doppelt verketteten Liste.



Dies kann für einige Anwendungen günstig sein. Insbesondere ist das Entfernen einfacher, da ein zu entfernendes Listenelement bereits den Pointer auf seinen Vorgänger und seinen Nachfolger enthält. Im Beispielprogramm `liste.c` wird damit in der Funktion `elementLoeschen()` die Suche nach dem Vorgänger überflüssig.

Um eine doppelt verkettete Liste rückwärts vom letzten Element aus zu durchlaufen, kann zusätzlich das letzte Element gespeichert werden, was dann allgemein als End-Pointer bezeichnet wird. Start- und End-Pointer werden häufig in einer Struktur, dem sogenannten „Listenkopf“, gespeichert.

Eine doppelt verkettete Liste kann aber auch zu einer Ringstruktur angeordnet werden:



Der Vorteil einer solchen Ringstruktur ist, dass man mittels des Ansprechens des Rückwärtspointers des ersten Elements mit nur einem Schritt auf das letzte Element zugreifen kann. So kann man selbst ohne Listenkopf vor- als auch rückwärts in der Liste suchen.

Auf ein Beispiel zu einer doppelt verketteten Liste wird an dieser Stelle verzichtet. Es wird jedoch in einer Übungsaufgabe darauf eingegangen. Es sei darauf hingewiesen, dass die Aktionen Einfügen und Entfernen mit Sorgfalt zu programmieren sind. Eine Handskizze der einzelnen Listenelemente hilft bei der Pointervielfalt oft weiter.

Als letzte Anmerkung soll noch die Idee des sogenannten „**Sentinels**“ angesprochen werden. Bei einer doppelt verketteten Liste, welche zu einer Ringstruktur verknüpft ist, kann man ein spezielles „nulltes“ Element definieren, das Sentinel. Dieses speichert als Vorwärts- und Rückwärts-Pointer einen Pointer auf das erste, beziehungsweise das letzte Element der Liste. Als Daten wird ein NULL-Pointer gespeichert. Anders gesagt, speichert man im „nullten“ Element den Listenkopf der Liste.

Mittels eines solchen Sentinels ist es möglich, eine Implementation zu generieren, welche fast gänzlich ohne Spezialfälle (sprich `if`-Strukturen) auskommt. Auch hierfür wird auf eine Übungsaufgabe am Ende dieses Kapitels verwiesen.

19.2 Baumstrukturen

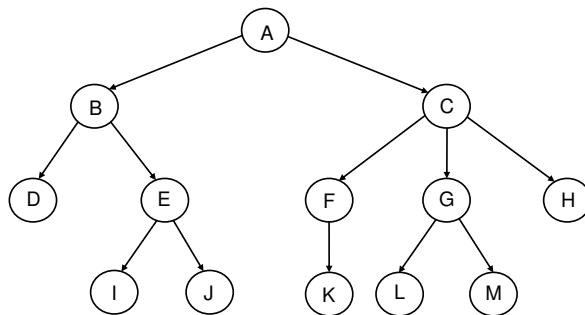
Ein bekanntes Beispiel für **Bäume** aus dem Alltag ist die Evolutionstheorie nach Darwin, wo jede Tiergattung evolutionsgeschichtlich aus einer anderen Gattung hervorging und gleichzeitig mehrere Untergattungen hervorbringen kann. Ein anderes Beispiel ist das Organigramm einer großen Firma, wo zuoberst der Chef als Wurzelknoten steht, darunter die Filialen, darunter die Abteilungen, darunter die Gruppen, und ganz zuunterst die Mitarbeiter.

Bäume als Datenstrukturen können sehr gut zum effizienten Abspeichern und Suchen mit Hilfe eines Schlüsselwertes eingesetzt werden.



19.2.1 Allgemeine Darstellung von Baumstrukturen

Zur Darstellung von Baumstrukturen gibt es verschiedene grafische und alphanumerische Möglichkeiten. In folgenden Bild wird ein Baum durch einen Graphen aus **Knoten** und **Kanten** dargestellt:



Die Darstellung der Knoten ist nicht genormt. Üblicherweise werden sie kreisförmig oder rechteckig dargestellt.

Wie man leicht in diesem Bild sieht, stehen in der Informatik die Bäume auf dem Kopf.

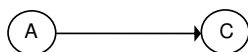
Der oberste Knoten heißt „**Wurzel**“. Die Knoten ohne Nachfolger bilden die „Blätter“ (**Blattknoten**) des Baumes. Alle anderen Knoten werden als „**innere Knoten**“ bezeichnet.



Im obigen Beispiel ist A die Wurzel, Knoten B, E, F, C und G sind innere Knoten und Knoten D, I, J, K, L, M und H sind die Blätter.

19.2.2 Formale Definition eines Baumes

Ein Baum ist ein sogenannt gerichteter, azyklischer Graph, auf Englisch auch DAG genannt (directed acyclic graph). Gerichtet bedeutet, dass die Verbindung zwischen Knoten stets von einem sogenannten „Elternknoten“ (näher an der Wurzel) zu einem „Kinds-knoten“ (näher bei den Blättern) zeigt. Im Beispiel ist A der Elternknoten vom Kind C.



In den üblichen Baumdarstellungen steht ein Elternknoten stets höher als das Kind. So wird damit implizit die Richtung der Kanten ausgedrückt und die Pfeilspitze an den Kanten kann dann der Einfachheit halber auch weggelassen werden.

Formal kann ein Baum folgendermaßen definiert werden:

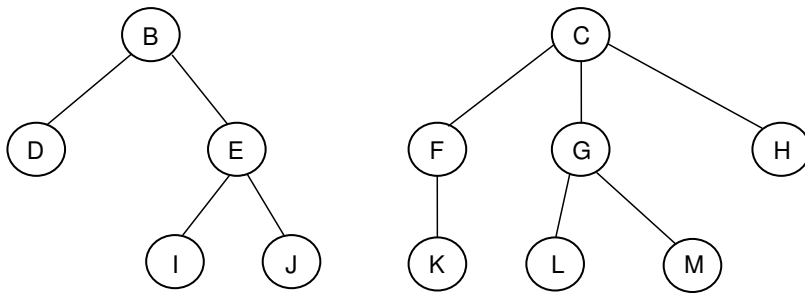
- Ein Baum besteht aus einer endlichen Anzahl Knoten, die durch gerichtete Kanten verbunden sind.
- Es gibt nur einen einzigen Knoten, der keine Beziehung zu einem Elternknoten hat. Dieser Knoten stellt die Wurzel dar.
- Alle übrigen Knoten haben genau einen Elternknoten.
- Ein Knoten kann eine beliebige endliche Zahl an Kinds-knoten haben.



Aus diesen Regeln folgt zudem: Es darf keine disjunkten Teile geben, das heißt alle Knoten müssen untereinander verknüpft sein. Es darf also kein loses Teil dabei sein.

Knoten, die denselben Elternknoten haben, werden als Geschwister bezeichnet.

Entfernt man die Wurzel eines Baumes, beispielsweise die Wurzel im vorangehenden Bild, so werden die Nachfolgerknoten zu den Wurzeln der entsprechenden Teilbäume (Unterbäume):



Diese Teilbäume sind disjunkt, bilden also insgesamt keinen gemeinsamen Baum, sind jedoch für sich selbst genommen jeweils wiederum eigenständige Bäume.

Ein Baum wird als „**geordneter Baum**“ bezeichnet, wenn die Reihenfolge der Knoten einer vorgeschriebenen Ordnung unterliegt.



In diesem Beispiel sind alle Knoten in alphabetischer Reihenfolge von oben nach unten und dann von links nach rechts sortiert.

Bei geordneten Bäumen spielt die Reihenfolge der Teilbäume eine wichtige Rolle. Würde man die Positionen der Teilbäume vertauschen, so würde man den Informationsgehalt des Baumes unzulässigerweise manipulieren.

Man kann den Begriff eines Baumes erweitern, wenn man auch leere Knoten im Baum zulässt. Leere Knoten sind Platzhalter, die bei Bedarf echte Knoten aufnehmen können.

Als spezieller Baum ergibt sich der leere Baum, der aus einer leeren Wurzel besteht und keinen einzigen echten Knoten hat.



19.2.3 Binäre Bäume

Binäre Bäume sind Bäume, für die gilt:

- Alle Knoten, die nicht leer sind, besitzen zwei Kinder.
- Maximal ein Kind eines Elternknotens kann leer sein.

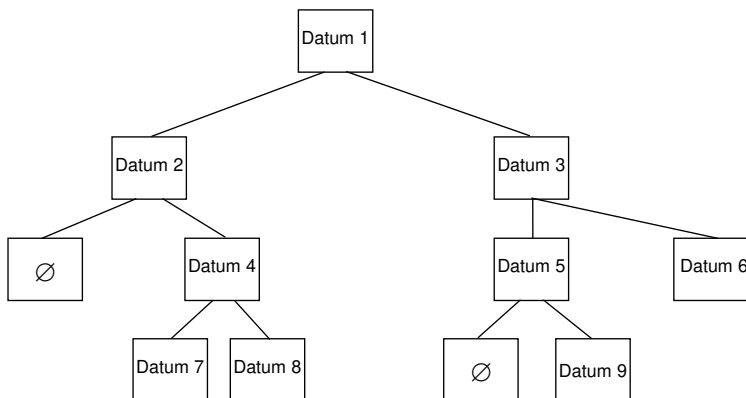
Rekursiv kann ein binärer Baum folgendermaßen definiert werden:

Ein binärer Baum

- ist entweder leer (leerer Baum)
- oder besteht aus einem einzigen Knoten
- oder besteht aus einer Wurzel, die einen linken und einen rechten binären Unterbaum hat, die beide nicht zugleich leere Bäume sein dürfen.



Das folgende Bild gibt ein Beispiel für einen binären Baum:



Die im Bild dargestellten Knoten mit dem Symbol $\emptyset$ stellen leere Knoten dar.

Binäre Bäume werden sehr häufig als geordnete Bäume programmiert. Dadurch ist – wie oben bereits angesprochen – die Reihenfolge der Teilbäume wichtig. Man kann in der Zeichnung die leeren Knoten weglassen, muss dann aber sicherstellen, dass aus der Anordnung der Knoten eindeutig hervorgeht, ob der rechte oder der linke Knoten leer ist.

Geordnete binäre Bäume sind in der Informatik allgegenwärtig. Beispielsweise eignen sie sich sehr gut zur Speicherung von Ausdrücken mit zweistelligen Operatoren, wobei jeder Operator einen Knoten mit den zugehörigen Operanden als Teilbäume darstellt. Weiterhin werden sie bei lexikalischen Sortierverfahren eingesetzt. In diesem Falle spricht man auch von binären Suchbäumen.

19.2.4 Binäre Suchbäume

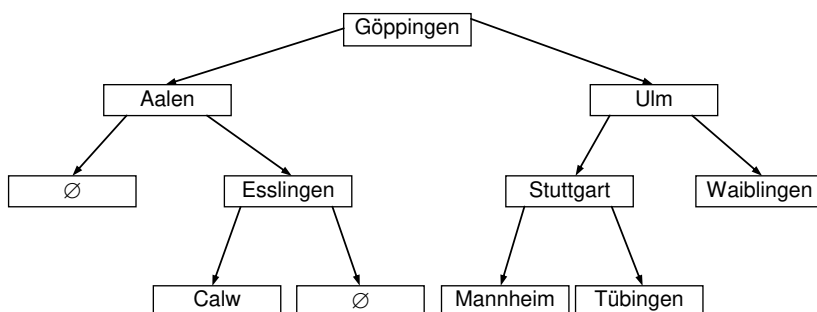
Binäre Suchbäume speichern eine Menge von Daten mit einem sogenannten „Schlüsselwert“, auf Englisch auch „key value“ genannt. Die Knoten werden so im Baum abgelegt, dass die Schlüsselwerte einer gegebenen Sortierung folgen. Dies wird beispielsweise folgendermaßen definiert:

Der Schlüsselwert des linken Teilbaums ist stets kleiner als der Schlüsselwert der Wurzel und der Schlüsselwert des rechten Teilbaumes ist stets größer als der Schlüsselwert der Wurzel.



Dabei bezeichnet der Schlüsselwert eines Teilbaumes den Schlüsselwert des Wurzelknotens des Teilbaumes.

Folgendes Bild zeigt ein Beispiel für einen solchen binären **Suchbaum**:



In diesem Baum werden Schlüsselwerte in Form eines Names von Ortschaften abgelegt. Wie man leicht nachprüfen kann, wurde der Baum geordnet aufgebaut. Die linken Teilbäume enthalten dabei Informationen, deren Schlüsselwerte kleiner als die Schlüsselwerte der Informationen der rechten Teilbäume sind. Kleiner bezieht sich hierbei auf die alphabetische Ordnung.

Der Vorteil einer solchen Ordnung zeigt sich beim Suchen und Sortieren.

19.2.5 Suchen in binären Suchbäumen

Um ein Element in einem binären Suchbaum zu finden, wird nach dem gewünschten Schlüsselwert gesucht.

Man vergleicht als erstes den Suchbegriff mit dem Schlüsselwert der Wurzel. Ist der Suchbegriff kleiner, so sucht man im linken Teilbaum weiter, ist er gleich, so hat man den gesuchten Knoten gefunden, ist er größer, so sucht man im rechten Teilbaum weiter.

Nach dieser Vorschrift sucht man rekursiv weiter, bis der Knoten gefunden ist. Wird er nicht gefunden, so ist der Suchbegriff im Baum nicht enthalten.

Die Höhe des Baumes (die maximale Anzahl der Ebenen) entspricht dabei der maximalen Anzahl von Vergleichen. In dem Beispiel mit den Ortsnamen werden maximal 4 Zugriffe benötigt, obschon 9 Ortschaften eingetragen sind. Bei größeren Datenmengen ist die Zeitersparnis noch bedeutend größer. Knoten können somit in binären Bäumen sehr schnell gesucht werden (daher der Name Suchbaum).

19.2.6 Einfügen und Entfernen von Elementen in binären Suchbäumen



Um Elemente in einen binären Suchbaum einzufügen wird zuerst nach dem Schlüsselwert gesucht. Sobald ein Knoten erreicht wird, bei welchem es keinen Nachfolger mehr gibt, so wird ein neuer Knoten rechts oder links von diesem Knoten erzeugt.

Um einen Knoten zu entfernen, wird zuerst ganz einfach nach dem Schlüsselwert des Knotens gesucht. Sofern es sich beim gefundenen Knoten um ein Blatt handelt (Knoten ohne Kinder), so wird einfach dieser Knoten entfernt.

Falls der gefundene Knoten jedoch noch ein oder zwei Teilbäume hat, so muss in dem zu entfernenden Knoten der Inhalt eines anderen, geeigneten Knotens gespeichert werden. Dies wird beispielsweise so gelöst, indem der Inhalt des links-äußersten Blatt-Knotens des rechten Teilbaumes oder der Inhalt des rechts-äußersten Blatt-Knotens des linken Teilbaumes an die Stelle des zu entfernenden Knotens gesetzt wird. So kann sichergestellt werden, dass sämtliche Knoten der beiden Teilbäume erneut garantiert entweder alle kleiner oder alle größer als der neue Wurzelknoten sind.

19.2.7 Durchlaufen von Bäumen

Zum Durchlaufen allgemeiner Bäume gibt es zwei verschiedene Durchlaufalgorithmen: Die Präfix-Ordnung und die Postfix-Ordnung. Bei binären Bäumen kommt noch die Infix-Ordnung dazu.

Eine Durchlaufordnung bezeichnet die Reihenfolge, in welcher die Eltern- und Kindsknoten abgearbeitet werden.



Bei der Präfix-Ordnung lautet die Reihenfolge:

1. Zuerst die Wurzel,
2. Dann alle Unterbäume in Präfix-Ordnung

Bei der Postfix-Ordnung lautet die Reihenfolge:

1. Zuerst alle Unterbäume in Postfix-Ordnung,
2. Dann die Wurzel.

Präfix- und Postfix-Ordnung sind bei der Implementierung von Sprachumwandlern von Bedeutung. Diese beiden Ordnungen erlauben die Umwandlung arithmetischer Ausdrücke, die als binäre Bäume dargestellt sind, in klammerlose Darstellungen – nämlich in die sogenannte Polnische beziehungsweise in die Umgekehrte Polnische Notation.



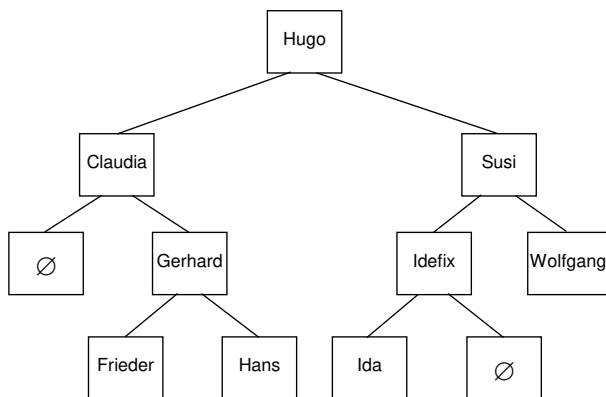
Bei binären Bäumen ist die Infix-Ordnung wichtig. Da alle Knoten eines binären Baumes zwei Kindsknoten besitzen, lautet die Reihenfolge:

1. Zuerst der linke Teilbaum in Infix-Ordnung,
2. Dann die Wurzel.
3. Dann der rechte Teilbaum in Infix-Ordnung.

Die Infix-Ordnung wird auch symmetrischer Durchlauf genannt.



Der folgende Baum soll in der durch diese drei verschiedenen Ordnungen festgelegten Reihenfolge durchlaufen werden.



Zur besseren Unterscheidung wird im Folgenden ein Knoten nur durch seinen Knotennamen und ein Unterbaum, der in einem Knoten beginnt, durch {Unterbaum Knotenname} dargestellt. Damit ergeben sich die folgenden Reihenfolgen für die Durchläufe durch die einzelnen Knoten:

Durchlauf in Präfix-Ordnung:

Hugo, {Unterbaum Claudia}, {Unterbaum Susi}
 = Hugo, Claudia, {Unterbaum Gerhard}, Susi, {Unterbaum Idefix}, Wolfgang
 = Hugo, Claudia, Gerhard, Frieder, Hans, Susi, Idefix, Ida, Wolfgang

Durchlauf in Postfix-Ordnung:

{Unterbaum Claudia}, {Unterbaum Susi}, Hugo
 = {Unterbaum Gerhard}, Claudia, {Unterbaum Idefix}, Wolfgang, Susi, Hugo
 = Frieder, Hans, Gerhard, Claudia, Ida, Idefix, Wolfgang, Susi, Hugo

Durchlauf in Infix-Ordnung:

{Unterbaum Claudia}, Hugo, {Unterbaum Susi}
 = Claudia, {Unterbaum Gerhard}, Hugo, {Unterbaum Idefix}, Susi, Wolfgang
 = Claudia, Frieder, Gerhard, Hans, Hugo, Ida, Idefix, Susi, Wolfgang

Druckt man also beispielsweise die Namen aller Knoten des Baumes aus, so kommt man je nach Durchlauf-Ordnung zu der soeben ermittelten Reihenfolge der Namen. Dabei ist zu beachten, dass die Infix-Ordnung aufgrund der Sortieranweisung des binären Suchbaumes automatisch zu einer alphabetischen Sortierung führt.

Würde man die Infix-Durchlaufordnung so ändern, dass zuerst der rechte und danach der linke Teilbaum durchlaufen wird, so erhält man die umgekehrte symmetrische Ordnung, sprich das Resultat entspricht einer umgekehrten alphabetischen Sortierung:

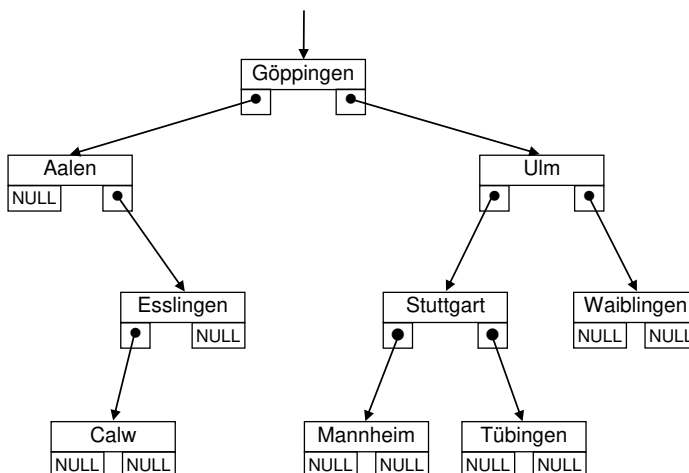
{Unterbaum Susi}, Hugo, {Unterbaum Claudia}
 = Wolfgang, Susi, {Unterbaum Idefix}, Hugo, {Unterbaum Gerhard}, Claudia
 = Wolfgang, Susi, Idefix, Ida, Hugo, Hans, Gerhard, Frieder, Claudia

Bei den hier vorgestellten Durchlaufalgorithmen läuft man zuerst tief die Äste des Baumes hinunter bis zu den Blättern. Man spricht deswegen auch von Tiefensuchverfahren (depth first search = DFS). Im Gegensatz dazu bearbeitet man bei dem Breitensuchverfahren (breadth first search = BFS) zuerst alle Knoten auf einer Ebene. Das bedeutet, man bearbeitet zuerst alle Kinder der Wurzel, dann alle Kinder dieser Kinder, usw.

Auf Algorithmen für Breitensuchverfahren und weitere Algorithmen für Tiefensuchverfahren kann an dieser Stelle nicht eingegangen werden.

19.2.8 Beispielprogramm für binäre Bäume

Im Folgenden wird ein binärer Baum mit Hilfe von Strukturen und Pointern auf Strukturen realisiert. Eine Struktur repräsentiert einen Knoten. Jede Struktur enthält zwei Pointer. Diese Pointer referenzieren dabei die Wurzel des linken beziehungsweise die Wurzel des rechten Teilbaums. Ist ein Teilbaum leer, so wird der NULL-Pointer als Pointer abgelegt. In einem Blatt wird folglich zweimal der NULL-Pointer abgespeichert.



Das folgende Programm baut einen binären Suchbaum auf, erlaubt es, einen Knoten im Baum zu suchen und druckt einen gesamten Teilbaum aus.

baum.c

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

struct Knoten {
    char str[30];
    int plz;
    struct Knoten* pointerLinks;
    struct Knoten* pointerRechts;
};

struct Knoten* wurzel = NULL;

struct Knoten* knotenEinfuegen(
    struct Knoten* ptr,
    const char* string,
    int plz)
{
    struct Knoten* neuPtr = NULL;

    if (ptr == NULL) {
        neuPtr = malloc(sizeof(struct Knoten));
        if (neuPtr != NULL) {
            strcpy(neuPtr->str, string) ;
            neuPtr->plz = plz;
            neuPtr->pointerLinks = NULL;
            neuPtr->pointerRechts = NULL;
        }
    } else if (strcmp(string, ptr->str) < 0) {
        neuPtr = knotenEinfuegen(ptr->pointerLinks, string, plz);
        if ((neuPtr != NULL) && (ptr->pointerLinks == NULL)) {
            ptr->pointerLinks = neuPtr;
        }
    } else if (strcmp(string, ptr->str) > 0) {
        neuPtr = knotenEinfuegen(ptr->pointerRechts, string, plz);
        if ((neuPtr != NULL) && (ptr->pointerRechts == NULL)) {
            ptr->pointerRechts = neuPtr;
        }
    } else {
        printf("Eintrag in Baum bereits vorhanden\n");
    }
}
```

```
    if (wurzel == NULL)
        wurzel = neuPtr;

    return neuPtr;
}

struct Knoten* suche(struct Knoten* ptr, const char* string) {
    if (ptr != NULL) {
        int comp = strcmp(string, ptr->str);
        if (comp < 0)
            return suche(ptr->pointerLinks, string);
        else if (comp > 0)
            return suche(ptr->pointerRechts, string);
        else
            return ptr;
    }
    return NULL;
}

void drucke(const struct Knoten* ptr) {
    if (ptr != NULL) {
        drucke(ptr->pointerLinks);
        printf("%s: %d\n", ptr->str, ptr->plz);
        drucke(ptr->pointerRechts);
    }
}

int main(void) {
    knotenEinfuegen(wurzel, "Goeppingen", 73033);
    knotenEinfuegen(wurzel, "Ulm", 89073);
    knotenEinfuegen(wurzel, "Aalen", 73430);
    knotenEinfuegen(wurzel, "Weiblingen", 71336);
    knotenEinfuegen(wurzel, "Stuttgart", 70173);
    knotenEinfuegen(wurzel, "Esslingen", 70771);
    knotenEinfuegen(wurzel, "Calw", 75365);
    knotenEinfuegen(wurzel, "Tuebingen", 72070);
    knotenEinfuegen(wurzel, "Mannheim", 68159);

    drucke(wurzel);
    printf("\n");

    struct Knoten* stuttgart = suche(wurzel, "Stuttgart");
    drucke(stuttgart);
    printf("\n");
    return 0;
}
```

Es ist zu beachten, dass das Löschen eines Elementes in diesem Beispielprogramm explizit weggelassen wurde. Es gibt hierfür eine Übungsaufgabe.

In der Funktion `main()` wird zuerst der gesamte Baum ausgegeben und danach der Teilbaum ab dem Ort „Stuttgart“. Die Ausgabe des Programmes lautet:

```
Aalen: 73430
Calw: 75365
Esslingen: 70771
Goepplingen: 73033
Mannheim: 68159
Stuttgart: 70173
Tuebingen: 72070
Ulm: 89073
Weiblingen: 71336

Mannheim: 68159
Stuttgart: 70173
Tuebingen: 72070
```

Die Funktion `suche()` beginnt die Suche eines Knotens in der Wurzel und vergleicht den Suchstring mit dem in der Wurzel gespeicherten String. Ist der Suchstring größer als der dort gespeicherte String, so wird zum rechten Kind gegangen und der dort gespeicherte String wird mit dem Suchstring verglichen. Dieses Verfahren wird fortgesetzt, bis der Knoten gefunden ist, beziehungsweise bis der ganze Baum durchlaufen ist. Wird der ganze Baum durchlaufen und der Knoten ist nicht gefunden, so ist er auch nicht im Baum enthalten.

Der Funktion `knotenEinfuegen()` wird ein Pointer `ptr` auf einen Knoten und die in dem anzulegenden Knoten zu speichernden Daten übergeben. Ist der übergebene Pointer der NULL-Pointer, so wird auf ein leeres Blatt gezeigt. Deshalb kann an dieser Stelle der neue Knoten erzeugt und in den Baum eingefügt werden.

Ist der übergebene Pointer von NULL verschieden, so ist der Platz, auf den der Pointer zeigt, durch einen Knoten besetzt. Der einzufügende String wird mit dem String des vorhandenen Knotens verglichen. Ist der einzufügende String kleiner, so wird durch Aufruf von `knotenEinfuegen()` zum linken Kind des vorhandenen Knotens gegangen, wobei der Pointer `ptr->pointerLinks`, an `knotenEinfuegen()` übergeben wird. Ist der einzufügende String größer, so wird in entsprechender Weise zum rechten Kind gegangen. Die Funktion `knotenEinfuegen()` wird also rekursiv aufgerufen.

Ist nun das Kind ein leeres Blatt, so wird das rekursive Aufrufen beendet. Es wird nun ein Knoten angelegt und in ihm die übergebenen Daten abgespeichert. Durch die Beendigung der Rekursion wird rückwärts durch die unterbrochenen Funktionen gegangen. Die einzige Funktion, die dabei noch Anweisungen abarbeitet, ist die Funktion, die `knotenEinfuegen()` mit dem NULL-Pointer aufgerufen hat. Sie erhält als Rückgabewert dieser Funktion den Pointer auf den neuen Knoten und kann damit den neu geschaffenen Knoten mit dem bestehenden Baum verketteten. Alle früher aufgerufenen Funktionen `knotenEinfuegen()` tun nichts weiteres, als den Pointer auf den neuen Knoten als Return-Wert an ihren Elternknoten durchzureichen.

19.3 Übungsaufgaben

Aufgabe 1: Einfach verkettete Liste

Betrachten Sie folgende Datenstruktur:

```
typedef struct Element Element;  
struct Element {  
    int x;  
};
```

- a) Schreiben Sie eine Funktion, welche ein Element im Speicher dynamisch reserviert und einen gegebenen int-Wert speichert.

```
Element* erstelleElement(int x);
```

- b) Erweitern Sie das Programm so, dass aus dieser Struktur eine einfach verkettete Liste aus Elementen erstellt werden kann. Speichern sie einen Pointer auf das Start-Element in einer globalen Variable. Schreiben Sie eine Funktion, um ein Element an den Anfang der Liste hinzuzufügen:

```
void hinzufuegenElement(Element* element);
```

- c) Schreiben Sie eine Ausgabe-Funktion, welche die Werte x einer Liste der Reihe nach ausgibt. Verwenden Sie dazu eine while-Schleife.

```
void ausgabeListe()
```

- d) Schreiben Sie eine Funktion, um ein Element ans Ende der Liste hinzuzufügen:

```
void hinzufuegenElementEnde(Element* element);
```

- e) Schreiben Sie eine zweite Ausgabe-Funktion, welche die Liste der Reihe nach ausgibt. Verwenden Sie diesmal dazu eine Rekursion.

```
void ausgabeListe2(Element* element)
```

- f) Schreiben Sie eine dritte Ausgabe-Funktion. Diesmal sollen die Werte jedoch in umgekehrter Reihenfolge ausgegeben werden. Nutzen sie dazu die Rekursion.

```
void ausgabeListe3(Element* element)
```

Aufgabe 2: Doppelt verkettete Liste

- a) Erweitern Sie die Struktur aus Aufgabe 1 so, dass sie nebst einem Pointer auf das nächste Element auch einen Pointer auf das vorherige Element besitzt.
- b) Schreiben Sie `hinzufuegenElement()` und `hinzufuegenElementEnde()` neu.
- c) Schreiben Sie eine neue Funktion, welche ein Element an einer beliebigen Stelle in der Liste **nach** einem gegebenen Element hinzufügt.

```
void hinzufuegenElementNach(  
    Element* element,  
    Element* vorheriges);
```

- d) Schreiben Sie eine weitere Ausgabe-Funktion, welche die Liste ebenfalls in umgekehrter Reihenfolge ausgibt, diesmal jedoch mit einer `while`-Schleife.

```
void ausgabeListe4(Element* element)
```

- e) **Schwierig:** Lesen Sie nochmals das Kapitel [→ 19.1.6](#) und bauen Sie ein Sentinel in die Liste ein. Das Sentinel soll bei einer Liste stets vorhanden sein, sprich auch bei einer leeren Liste. Die Vorwärts- und Rückwärts-Pointer zeigen bei einer leeren Liste auf das Sentinel selbst. Versuchen Sie, die Implementation aller Funktionen zu vereinfachen.